

Lessons

REVIEWS ABOUT US **START LEARNING** 

3 MONTHS

JAVA FUNDAMENTALS
ONLINE COURSE WITH A MENTOR

 Training in groups

 Online lessons with experienced mentors

 Practical homework

 Additional tasks and coding projects

MORE DETAILS

Hurry up, there are only few places left in the group!

[My Progress](#) [Courses](#) [University](#) [Quest Map](#) [Lessons](#)

[CodeGym](#) / [Courses](#) / [All lectures for EN purposes](#) / [Related project: SQL, JDBC and Hibernate](#)

Related project: SQL, JDBC and Hibernate

All lectures for EN purposes
Level 1 , Lesson 105

AVAILABLE

Today we will do the final project on the fourth JRU module. What will it be? Let's try to work with different technologies: MySQL, Hibernate, Redis, Docker. Now more subject.

Task: we have a relational MySQL database with a schema (country-city, language by country). And there is a frequent request of the city, which slows down. We came up with a solution - to move all the data that is requested frequently to Redis (in memory storage of the key-value type).

And we need not all the data that is stored in MySQL, but only a selected set of fields. The project will be in the form of a tutorial. That is, here we will raise the problem and immediately solve it.

So, let's start with what software we will need:

1. IDEA Ultimate (who ran out of the key - write to Roman in the Slack)
2. Workbench (or any other client for MySQL)
3. [Docker](#)
4. [redis-insight](#) - optional

Our action plan:

1. Set up docker (I won't do this in the tutorial, because each OS will have its own characteristics and there are a lot of answers on the Internet to questions like "how to install docker on windows"), check that everything works.
2. Run MySQL server as a docker container.
3. Expand [dump](#) .
4. Create a project in Idea, add maven dependencies.
5. Make layer domain.
6. Write a method to get all data from MySQL.

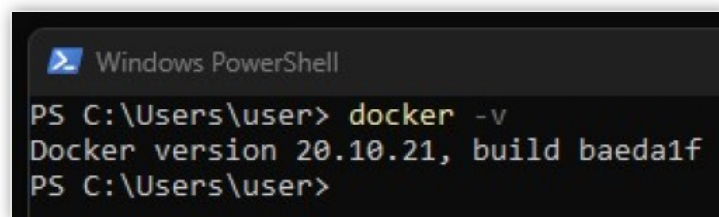
7. Write a data transformation method (in Redis we will write only the data that is requested frequently).
8. Run the Redis server as a docker container.
9. Write data to Redis.
10. Optional: install redis-insight, look at the data stored in Redis.
11. Write a method for getting data from Redis.
12. Write a method for getting data from MySQL.
13. Compare the speed of getting the same data from MySQL and Redis.

Docker setup

Docker is an open platform for developing, delivering and operating applications. We will use it in order not to install and configure Redis on the local machine, but to use a ready-made image. You can read more about docker [here](#) or see it [here](#) . If you are not familiar with docker, I recommend looking at just the second link.

To make sure you have docker installed and configured, run the command: `docker -v`

If everything is OK, you will see the docker version



```
Windows PowerShell
PS C:\Users\user> docker -v
Docker version 20.10.21, build baeda1f
PS C:\Users\user>
```

Run MySQL server as docker container

In order to be able to compare the time of returning data from MySQL and Redis, we will also use MySQL in the docker. In PowerShell (or another console terminal if you're not using Windows), run the command:

```
docker run --name mysql -d -p 3306:3306 -e MYSQL_ROOT_PASSWORD=root --restart unless-stopped
```

Consider what we are doing with this command:

- `docker run` - launching (and downloading, if it has not yet been downloaded to the local machine) the image. As a result of the launch, we get a running container.
- `--name mysql` - set the name of the mysql container.
- `-d` - a flag that says that the container should continue to work, even if you close the terminal window from which this container was launched.
- `-p 3306:3306` - specifies ports. Before the colon - the port on the local machine, after the colon - the port in the container.
- `-e MYSQL_ROOT_PASSWORD=root` - passing the environment variable `MYSQL_ROOT_PASSWORD` with the value `root` to the container. Flag specific to the mysql/ image
- `--restart unless-stopped` - setting the behavior policy (whether the container should

be restarted when closed). The unless-stopped value means to always restart, except when the container was stopped /

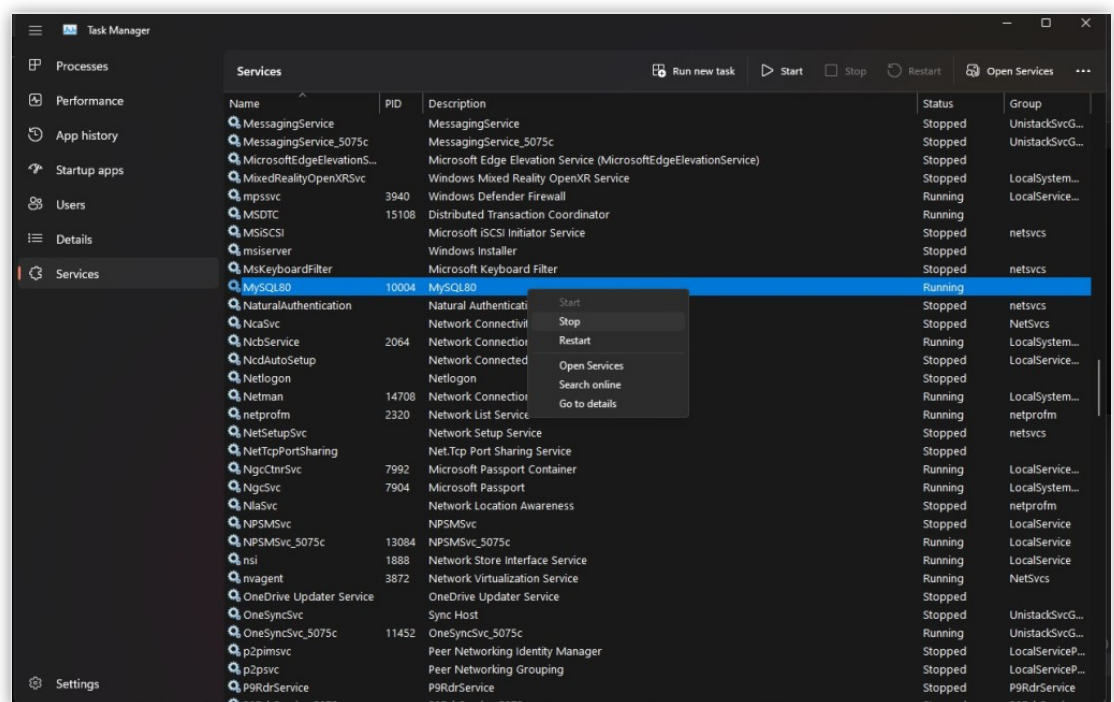
- `-v mysql:/var/lib/mysql` — add volume (image for storing information).
- `mysql:8` — the name of the image and its version.

After executing the command in the terminal, the docker will download all the layers of the image and start the container:

```

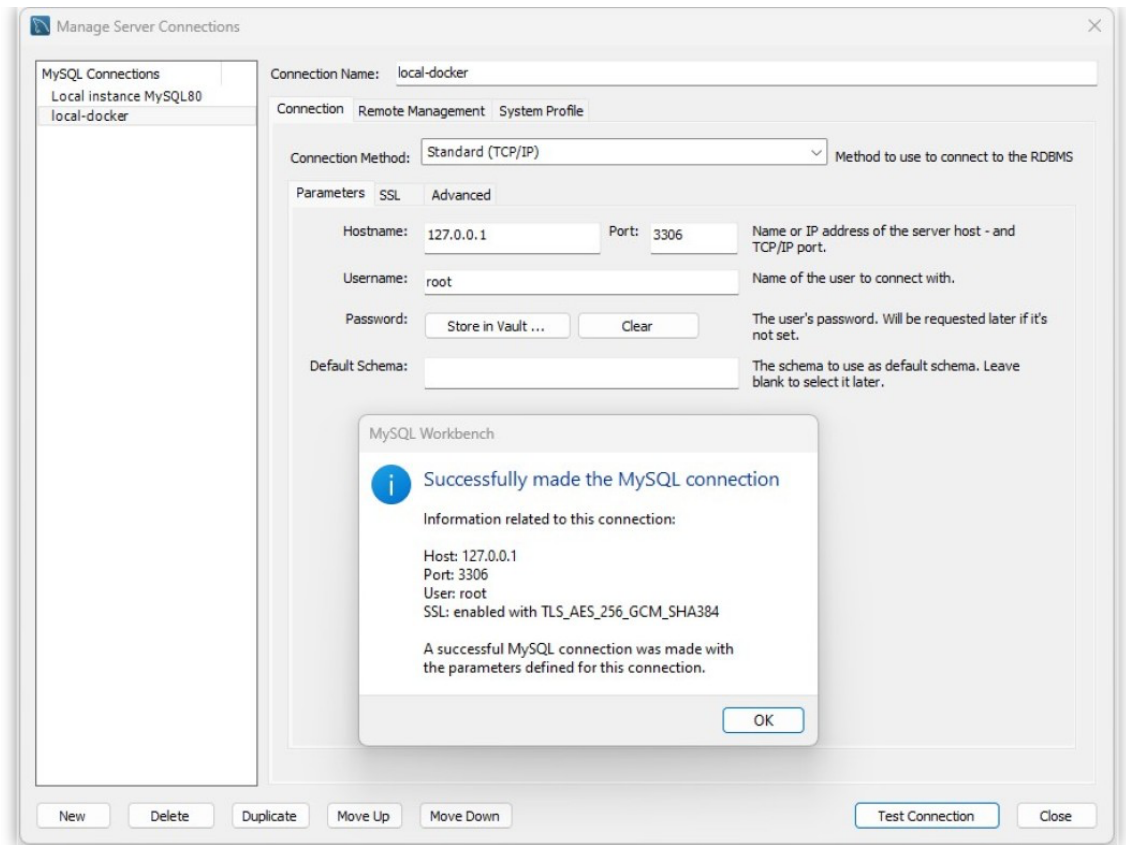
PS C:\Users\user> docker run --name mysql -d -p 3306:3306 -e MYSQL_ROOT_PASSWORD=root --restart unless-stopped -v mysql:/var/lib/mysql mysql:8
Unable to find image 'mysql:8' locally
8: Pulling from library/mysql
0ed027b72ddc: Pull complete
0296159747f1: Pull complete
3d2f9b664bd3: Pull complete
df6519f81c26: Pull complete
36bb5e56d458: Pull complete
054e8fde88d0: Pull complete
f2b494c59c7f: Pull complete
132bc0d471b8: Pull complete
135ec7033a05: Pull complete
5961f0272472: Pull complete
75b5f7a3d3a4: Pull complete
Digest: sha256:3d7ae561cf6095f6aca8eb7830e1d14734227b1fb4748092f2be2cfbccf7d614
Status: Downloaded newer image for mysql:8
9c10ec4be24c332ed49259e0a0a9dd712fc3f0f0a438d092e385a765de1f750e
PS C:\Users\user>
  
```

Important note: if you have MySQL installed as a service on your local computer and it is running, you need to specify a different port in the start command, or stop this running service.



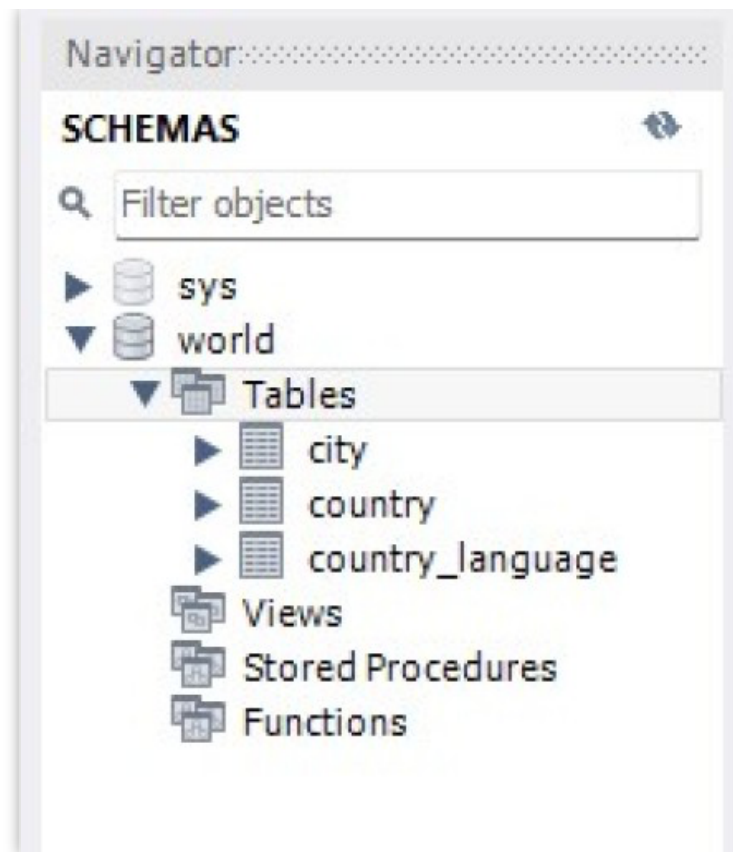
Expand dump

To expand the dump, you need to create a new connection to the database from Workbench, where you specify the parameters. I used the default port (3306), didn't change the username (root by default), and set the password for the root user (root).



In Workbench, do Data Import/Restore and select Import from Self Contained File . [Specify where you downloaded the dump](#) as a file . You don't need to create the schema beforehand - its creation is included in the dump file. After a successful import, you will have a world schema with three tables:

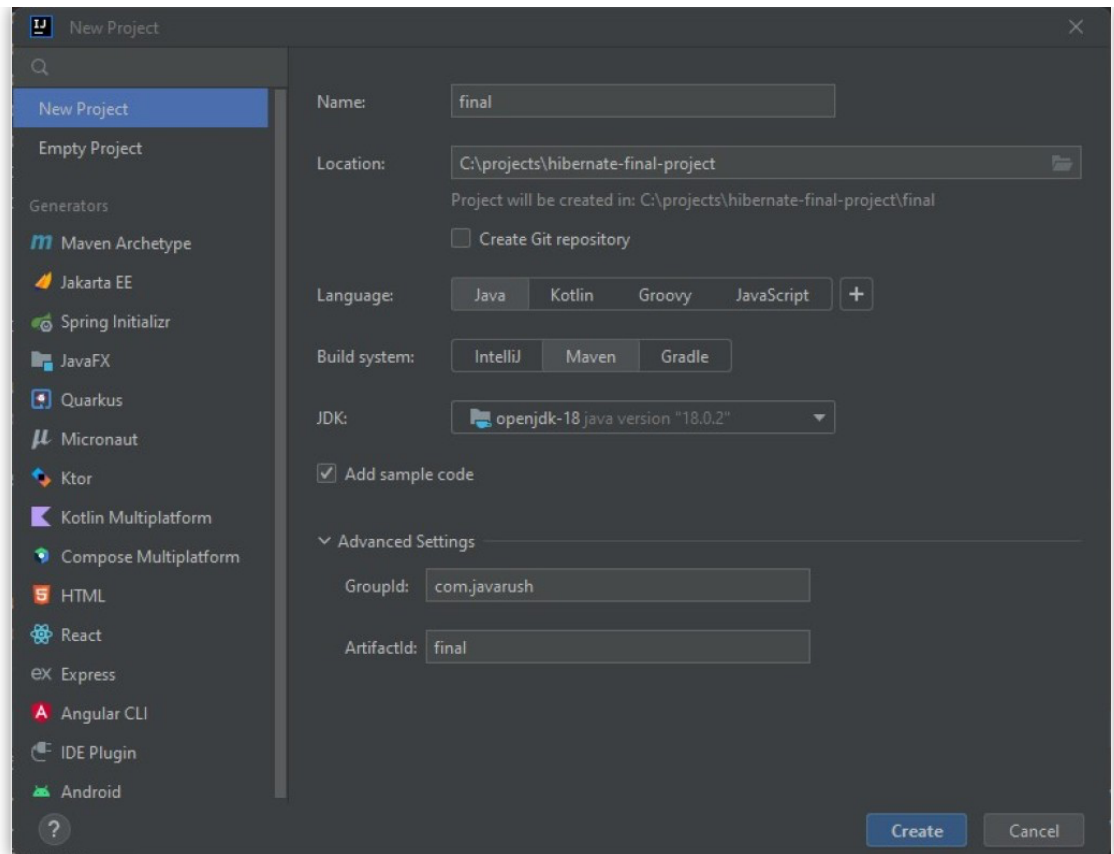
1. city is a table of cities.
2. country - country table.
3. country_language - a table that indicates what percentage of the population in the country speaks a particular language.



Since we used a volume when starting the container, after stopping and even deleting the mysql container and re-executing the start command (`docker run --name mysql -d -p 3306:3306 -e MYSQL_ROOT_PASSWORD=root --restart unless-stopped -v mysql:/var/lib/mysql mysql:8`), there will be no need to deploy the dump again - it is already deployed in the volume.

Create project in Idea, add maven dependencies

You already know how to create a project in the Idea - this is the easiest point in today's project.



Add dependencies to the pom file:

```
<dependencies>
  <dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>8.0.30</version>
  </dependency>

  <dependency>
    <groupId>org.hibernate</groupId>
    <artifactId>hibernate-core-jakarta</artifactId>
    <version>5.6.14.Final</version>
  </dependency>

  <dependency>
    <groupId>p6spy</groupId>
    <artifactId>p6spy</artifactId>
    <version>3.9.1</version>
  </dependency>

  <dependency>
    <groupId>io.lettuce</groupId>
    <artifactId>lettuce-core</artifactId>
    <version>6.2.2.RELEASE</version>
  </dependency>
</dependencies>
```

```
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-databind</artifactId>
  <version>2.14.0</version>
</dependency>
</dependencies>
```

 **Become
a Java Developer**

on an online course with a mentor 

and certification 

 Online lectures with experienced mentors

 Group training and private chat support

 10 projects in your portfolio, and hundreds of hours of coding

 Help you find your first job

Classes will start soon in a new group - hurry up!

[Learn more](#) 

The first three dependencies have long been familiar to you.

`lettuce-core` is one of the available Java clients for working with Redis.

`jackson-databind` - dependency for using ObjectMapper (to transform data for storage in Redis (key-value of type String)).

Also in the resources folder (src/main/resources) add spy.properties to view the requests with parameters that Hibernate executes. File contents:

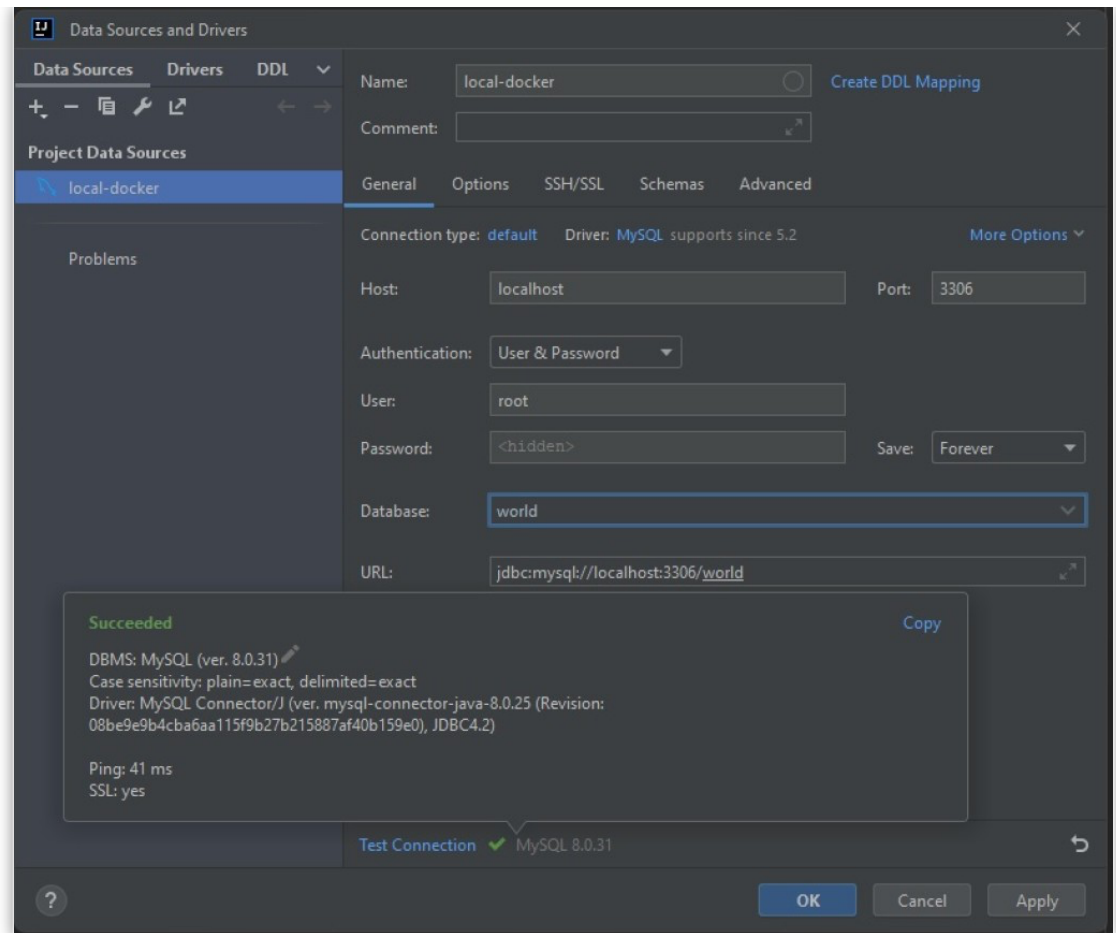
```
driverlist=com.mysql.cj.jdbc.Driver
dateformat=yyyy-MM-dd hh:mm:ss a
appender=com.p6spy.engine.spy.appender.StdoutLogger
logMessageFormat=com.p6spy.engine.spy.appender.MultiLineFormat
```

Make layer domain

Create package com.codegym.domain

It is convenient for me when mapping tables on an entity to use the table structure in the Idea, so let's add a database connection in the Idea.





I suggest creating entities in this order:

- Country
- City
- CountryLanguage

It is desirable that you perform the mapping yourself.

Country class code:

```
1 package com.codegym.domain;
2
3 import jakarta.persistence.*;
4
5 import java.math.BigDecimal;
6 import java.util.Set;
7
8 @Entity
9 @Table(schema = "world", name = "country")
10 public class Country {
11     @Id
12     @Column(name = "id")
13     private Integer id;
14
15     private String code;
```

```
16         @Column(name = "code_2")
17         private String alternativeCode;
18
19         private String name;
20
21         @Column(name = "continent")
22         @Enumerated(EnumType.ORDINAL)
23         private Continent continent;
24
25         private String region;
26
27         @Column(name = "surface_area")
28         private BigDecimal surfaceArea;
29
30         @Column(name = "indep_year")
31         private Short independenceYear;
32
33         private Integer population;
34
35         @Column(name = "life_expectancy")
36         private BigDecimal lifeExpectancy;
37
38         @Column(name = "gnp")
39         private BigDecimal GNP;
40
41         @Column(name = "gnpo_id")
42         private BigDecimal GNPOId;
43
44         @Column(name = "local_name")
45         private String localName;
46
47         @Column(name = "government_form")
48         private String governmentForm;
49
50
51         @Column(name = "head_of_state")
52         private String headOfState;
53
54         @OneToOne
55         @JoinColumn(name = "capital")
56         private City city;
57
58         @OneToMany(fetch = FetchType.EAGER)
59         @JoinColumn(name = "country_id")
60         private Set<CountryLanguage> languages;
61
62
63         //Getters and Setters omitted
64     }
```

65

There are 3 interesting points in the code.

The first is the **Continent** `enum`, which is stored in the database as ordinal values. In the structure of the country table, in the comments to the continent field, you can see which numerical value corresponds to which continent.

```

1  package com.codegym.domain;
2
3  public enum Continent {
4      ASIA,
5      EUROPE,
6      NORTH_AMERICA,
7      AFRICA,
8      OCEANIA,
9      ANTARCTICA,
10     SOUTH_AMERICA
11 }
```

The second point is a set of entities `CountryLanguage`. Here's a link `@OneToMany` that wasn't in the second draft of this module. By default, Hibernate will not pull the value of this set when requesting a country entity. But since we need to subtract all values from the relational database for caching, the `FetchType.EAGER`.

The third is the `city` field. Communication `@OneToOne` - like everything is familiar and understandable. But, if we look at the foreign key structure in the database, we see that the country (country) has a link to the capital (city), and the city (city) has a link to the country (country). There is a cyclical relationship.

We won't do anything with this yet, but when we get to the "Write a method for getting all data from MySQL" item, let's see what queries Hibernate executes, look at their number, and remember this item. Hibernate

City class code:

```

1  package com.codegym.domain;
2
3  import jakarta.persistence.*;
4
5  @Entity
6  @Table(schema = "world", name = "city")
7  public class City {
8      @Id
9      @GeneratedValue(strategy = GenerationType.IDENTITY)
10     private Integer id;
11
12     private String name;
13 }
```

```
14     @ManyToOne
15     @JoinColumn(name = "country_id")
16     private Country country;
17
18     private String district;
19
20     private Integer population;
21
22     //Getters and Setters omitted
23
24 }
25
```

CountryLanguage class code:

```
1  package com.codegym.domain;
2
3  import jakarta.persistence.*;
4  import org.hibernate.annotations.Type;
5
6  import java.math.BigDecimal;
7
8  @Entity
9  @Table(schema = "world", name = "country_language")
10 public class CountryLanguage {
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     @Column(name = "id")
14     private Integer id;
15
16     @ManyToOne
17     @JoinColumn(name = "country_id")
18     private Country country;
19
20     private String language;
21
22     @Column(name = "is_official", columnDefinition = "BIT")
23     @Type(type = "org.hibernate.type.NumericBooleanType")
24     private Boolean isOfficial;
25
26     private BigDecimal percentage;
27
28     //Getters and Setters omitted
29
30 }
```

Write a method to get all data from MySQL

In the Main class, we declare the fields:

```
1 private final SessionFactory sessionFactory;
2 private final RedisClient redisClient;
3
4 private final ObjectMapper mapper;
5
6 private final CityDAO cityDAO;
7 private final CountryDAO countryDAO;
```

and initialize them in the constructor of the Main class:

```
1 public Main() {
2     sessionFactory = prepareRelationalDb();
3     cityDAO = new CityDAO(sessionFactory);
4     countryDAO = new CountryDAO(sessionFactory);
5
6     redisClient = prepareRedisClient();
7     mapper = new ObjectMapper();
8 }
```

As you can see, there are not enough methods and classes - let's write them.

Declare a package com.codegym.dao and add 2 classes to it:

```
1 package com.codegym.dao;
2
3 import com.codegym.domain.Country;
4 import org.hibernate.SessionFactory;
5 import org.hibernate.query.Query;
6
7 import java.util.List;
8
9 public class CountryDAO {
10     private final SessionFactory sessionFactory;
11
12     public CountryDAO(SessionFactory sessionFactory) {
13         this.sessionFactory = sessionFactory;
14     }
15
16     public List<Country> getAll() {
17         Query<Country> query = sessionFactory.getCurrentSession()
18             .createQuery("select c from Country c");
19         return query.list();
20     }
21 }
```

```
1 package com.codegym.dao;
2
3 import com.codegym.domain.City;
4 import org.hibernate.SessionFactory;
```

```

5      import org.hibernate.query.Query;
6
7      import java.util.List;
8
9      public class CityDAO {
10         private final SessionFactory sessionFactory;
11
12         public CityDAO(SessionFactory sessionFactory) {
13             this.sessionFactory = sessionFactory;
14         }
15
16         public List<City> getItems(int offset, int limit) {
17             Query<City> query = sessionFactory.getCurrentSession().createQuery("from City");
18             query.setFirstResult(offset);
19             query.setMaxResults(limit);
20             return query.list();
21         }
22
23         public int getTotalCount() {
24             Query<Long> query = sessionFactory.getCurrentSession().createQuery("select count(*) from City");
25             return Math.toIntExact(query.uniqueResult());
26         }
27     }

```

Now you can import these 2 classes into Main. Still missing two methods:

```

1      private SessionFactory prepareRelationalDb() {
2          final SessionFactory sessionFactory;
3          Properties properties = new Properties();
4          properties.put(Environment.DIALECT, "org.hibernate.dialect.MySQLDialect");
5          properties.put(Environment.DRIVER, "com.mysql.jdbc.Driver");
6          properties.put(Environment.URL, "jdbc:mysql://localhost:3306/hibernate_test");
7          properties.put(Environment.USER, "root");
8          properties.put(Environment.PASS, "root");
9          properties.put(Environment.CURRENT_SESSION_CONTEXT_CLASS, "thread");
10         properties.put(Environment.HBM2DDL_AUTO, "validate");
11         properties.put(Environment.STATEMENT_BATCH_SIZE, "100");
12
13         sessionFactory = new Configuration()
14             .addAnnotatedClass(City.class)
15             .addAnnotatedClass(Country.class)
16             .addAnnotatedClass(CountryLanguage.class)
17             .addProperties(properties)
18             .buildSessionFactory();
19         return sessionFactory;
20     }

```



GYM



CodeGym University

We have not yet reached the radish, so the implementation of the initialization of the radish client

Learning

-  Courses
-  Tasks
-  Surveys & Quizzes
-  Games
-  Help
-  Schedule

Community

-  Users
-  Forum
-  Chat
-  Articles
-  Success stories
-  Activity
-  Reviews

Subscriptions

will remain a stub for now:

```
1 private void shutdown() {
2     if (nonNull(sessionFactory)) {
3         sessionFactory.close();
4     }
5     if (nonNull(redisClient)) {
6         redisClient.shutdown();
7     }
8 }
```

Finally, we can write a method in which we pull out all the cities:

```
1 private List<City> fetchData(Main main) {
2     try (Session session = main.sessionFactory.getCurrentSession()) {
3         List<City> allCities = new ArrayList<>();
4         session.beginTransaction();
5
6         int totalCount = main.cityDAO.getTotalCount();
7         int step = 500;
8         for (int i = 0; i < totalCount; i += step) {
9             allCities.addAll(main.cityDAO.getItems(i, step));
10        }
11        session.getTransaction().commit();
12        return allCities;
13    }
14 }
```

The implementation feature is such that we get 500 cities each. This is necessary because there are restrictions on the amount of transmitted data. Yes, in our case, we will not get to them, because we have a total of 4079 cities in the database. But in production applications, when you need to get a lot of data, this technique is often used.

And the implementation of the main method:

```
1 public static void main(String[] args) {
2     Main main = new Main();
3     List<City> allCities = main.fetchData(main);
4     main.shutdown();
5 }
```

Now we can run our application in debug for the first time and see how it works (or does not work - yes, it often happens).


```

38
39 public static void main(String[] args) { args: []
40     Main main = new Main(); main: Main@6131
41     List<City> allCities = main.fetchData(main); allCities: size = 4079
42     main.shutdown(); main: Main@6131
43 }
44
1 usage
45 private SessionFactory prepareRelationalDb() {
46     final SessionFactory sessionFactory;
47     Properties properties = new Properties();
48     properties.put(Environment.DIALECT, "org.hibernate.dialect.MySQL8Dialect");

```

Evaluate expression (Enter) or add a watch (Ctrl+Shift+Enter)

- args = {String[0]@6130} []
- main = {Main@6131}
- allCities = {ArrayList@6132} size = 4079
 - 0 = {City@6135}
 - id = {Integer@6235} 1
 - name = "Kabul"
 - country = {Country@6237}
 - id = {Integer@6240} 2
 - code = "AFG"
 - alternativeCode = "AF"
 - name = "Afghanistan"
 - continent = {Continent@6244} "ASIA"
 - region = "Southern and Central Asia"
 - surfaceArea = {BigDecimal@6246} "652090.00"
 - independenceYear = {Short@6247} 1919
 - population = {Integer@6248} 22720000
 - lifeExpectancy = {BigDecimal@6249} "45.9"
 - GNP = {BigDecimal@6250} "5976.00"
 - GNPOld = null
 - localName = "Afganistan/Afqanestan"
 - governmentForm = "Islamic Emirate"
 - headOfState = "Mohammad Omar"
 - city = {City@6135}
 - languages = {PersistentSet@6254} size = 5
 - 0 = {CountryLanguage@7771}
 - id = {Integer@7776} 7
 - country = {Country@6237}
 - language = "Pashto"
 - isOfficial = {Boolean@7778} true
 - percentage = {BigDecimal@7779} "52.4"
 - 1 = {CountryLanguage@7772}
 - id = {Integer@7781} 5
 - country = {Country@6237}
 - language = "Balochi"

Cities are getting. Each city gets a country, if it has not been previously subtracted from the database for another city. Let's roughly calculate how many queries Hibernate will send to the database:

- 1 request to find out the total number of cities (needed to iterate over 500 cities in order to know when to stop).
- $4079 / 500 = 9$ requests (list of cities).
- Each city gets a country, if it has not been subtracted earlier. Since there are 239 countries in the database, this will give us 239 queries.

Total 249 requests. And we also said that together with the country we would immediately receive a set of languages, otherwise there would be darkness in general. But it's still a lot, so let's tweak the behavior a bit. Let's start with reflections: what to do, where to run? But seriously - why are there so many requests. If you look at the request log, we see that each country is requested

separately, so the first simple solution: let's request all the countries together, because we know in advance that we will need all of them in this transaction.

In the `fetchData()` method, immediately after the start of the transaction, add the following line:

```
List<Country> countries = main.countryDAO.getAll();
```

We count requests:

- 1 - get all countries
- 239 - query for each country of its capital
- 1 - request for the number of cities
- 9 - request for lists of cities

Total 250. The idea is good, but it didn't work. The problem is that the country has a connection with the capital (city) `@OneToOne`. And such a link is loaded immediately by default (`FetchType.EAGER`). Let's put `FetchType.LAZY`, because anyway, we will load all the cities later in the same transaction.

```
1 @OneToOne(fetch = FetchType.LAZY)
2 @JoinColumn(name = "capital")
3 private City city;
```

Capitals are no longer requested separately, but the number of requests has not changed. Now, for each country, the **CountryLanguage** list is requested by a separate query. That is, there is progress, and we are moving in the right direction. If you remember, the lectures suggested the “join fetch” solution in order to request an entity with dependent data in one request by adding an additional join to the request. In **CountryDAO**, rewrite the HQL query in the method `getAll()` to:

```
"select c from Country c join fetch c.languages"
```

Launch. We look at the log, count requests:

- 1 - all countries with languages
- 1 - number of cities
- 9 - lists of cities.

Total 11 (we succeeded)) If you not only read all this text, but also tried to run it after each step of tuning the application, you should even visually note the acceleration of the entire application several times.

Write a data transformation method

Let's create a package `com.codegym.redis` in which we add 2 classes: **CityCountry** (data on the city and the country in which this city is located) and **Language** (data on the language). Here are all the fields that are often requested “by task” in the “braking request”.

```
1 package com.codegym.redis;
2
3 import com.codegym.domain.Continent;
4
5 import java.math.BigDecimal;
6 import java.util.Set;
7
8 public class CityCountry {
9     private Integer id;
10
11     private String name;
12
13     private String district;
14
15     private Integer population;
16
17     private String countryCode;
18
19     private String alternativeCountryCode;
20
21     private String countryName;
22
23     private Continent continent;
24
25     private String countryRegion;
26
27     private BigDecimal countrySurfaceArea;
28
29     private Integer countryPopulation;
30
31     private Set<Language> languages;
32
33     //Getters and Setters omitted
34 }
```

```
1 package com.codegym.redis;
2
3 import java.math.BigDecimal;
4
5 public class Language {
6     private String language;
7     private Boolean isOfficial;
8     private BigDecimal percentage;
9
10     //Getters and Setters omitted
11 }
```

In the main method, after getting all the cities, add the line

```
List<CityCountry>> preparedData = main.transformData(allCities);
```

And implement this method:

```
1  private List<CityCountry> transformData(List<City> cities) {
2      return cities.stream().map(city -> {
3          CityCountry res = new CityCountry();
4          res.setId(city.getId());
5          res.setName(city.getName());
6          res.setPopulation(city.getPopulation());
7          res.setDistrict(city.getDistrict());
8
9          Country country = city.getCountry();
10         res.setAlternativeCountryCode(country.getAlternativeCode());
11         res.setContinent(country.getContinent());
12         res.setCountryCode(country.getCode());
13         res.setCountryName(country.getName());
14         res.setCountryPopulation(country.getPopulation());
15         res.setCountryRegion(country.getRegion());
16         res.setCountrySurfaceArea(country.getSurfaceArea());
17         Set<CountryLanguage> countryLanguages = country.getLanguages();
18         Set<Language> languages = countryLanguages.stream().map(c
19             Language language = new Language();
20             language.setLanguage(cl.getLanguage());
21             language.setOfficial(cl.getOfficial());
22             language.setPercentage(cl.getPercentage());
23             return language;
24         }).collect(Collectors.toSet());
25         res.setLanguages(languages);
26
27         return res;
28     }).collect(Collectors.toList());
29 }
```

I think this method is self-explanatory: we just create a **CityCountry** entity and fill it with data from **City** , **Country** , **CountryLanguage** .

Run Redis server as a docker container

There are 2 options here. If you do the optional step "install redis-insight, look at the data stored in Redis", then the command is for you:

```
docker run -d --name redis-stack -p 6379:6379 -p 8001:8001 redis/redis-
```

If you decide to skip this step, then just:

```
docker run -d --name redis -p 6379:6379 redis:latest
```

The difference is that in the first option, port 8001 is forwarded to the local machine, to which you can connect with an external client to see what is stored inside. And I used to give meaningful names, therefore, `redis-stack` or `redis`.

After launch, you can see the list of running containers. To do this, run the command:

```
docker container ls
```

And you will see something like this:

```

PS C:\Users\user> docker container ls
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
dbd78a642365   redis/redis-stack:latest            "/entrypoint.sh"        28 seconds ago Up 26 seconds 0.0.0.0:6379->6379/tcp, 0.0.0.0:8001->8001/tcp  redis-stack
ed0bfb0f154e   mysql:8                             "docker-entrypoint.s..." 4 hours ago    Up 7 minutes  0.0.0.0:3306->3306/tcp, 33060/tcp    mysql
PS C:\Users\user>

```

If you need to find some command, you can either look at the help in the terminal (`docker help`) or google "how to ..." (for example, docker how to remove running container).

And we also called the initialization of the radish client in the Main constructor, but did not implement the method itself. Add implementation:

```

1  private RedisClient prepareRedisClient() {
2      RedisClient redisClient = RedisClient.create(RedisURI.create(
3          try (StatefulRedisConnection<String, String> connection = red
4              System.out.println("\nConnected to Redis\n");
5          }
6          return redisClient;
7      }

```

sout was added for educational purposes so that in the launch log you can see that everything is OK and the connection through the radish client passed without errors.

Write data to Redis

Add a call to the main method

```
main.pushToRedis(preparedData);
```

With this method implementation:

```

1  private void pushToRedis(List<CityCountry> data) {
2      try (StatefulRedisConnection<String, String> connection = red
3          RedisStringCommands<String, String> sync = connection.syn
4          for (CityCountry cityCountry : data) {
5              try {
6                  sync.set(String.valueOf(cityCountry.getId()), map
7              } catch (JsonProcessingException e) {
8                  e.printStackTrace();
9              }

```

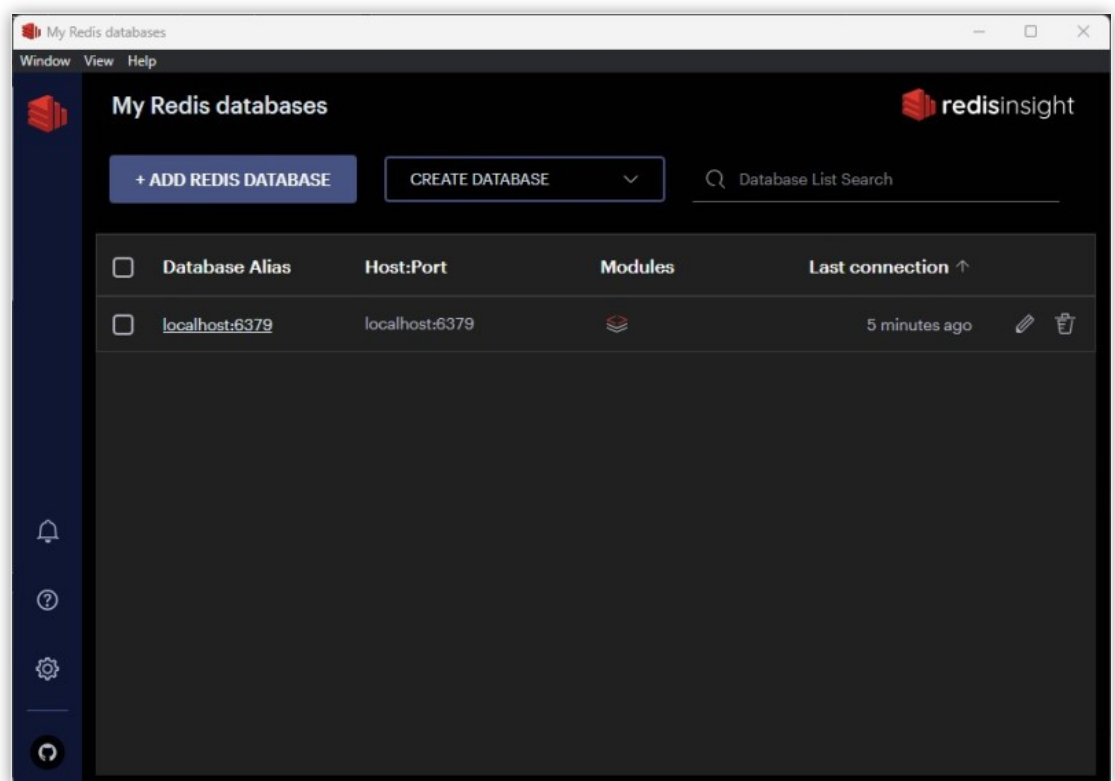
```
10         }  
11  
12     }  
13 }
```

Here, a synchronous connection is opened with the radish client, and sequentially each object of the **CityCountry** type is written to the radish. Since the radish is a **String** key-value store, the key (city id) is converted to a string. And the value is also to the string, but using **ObjectMapper** in JSON format.

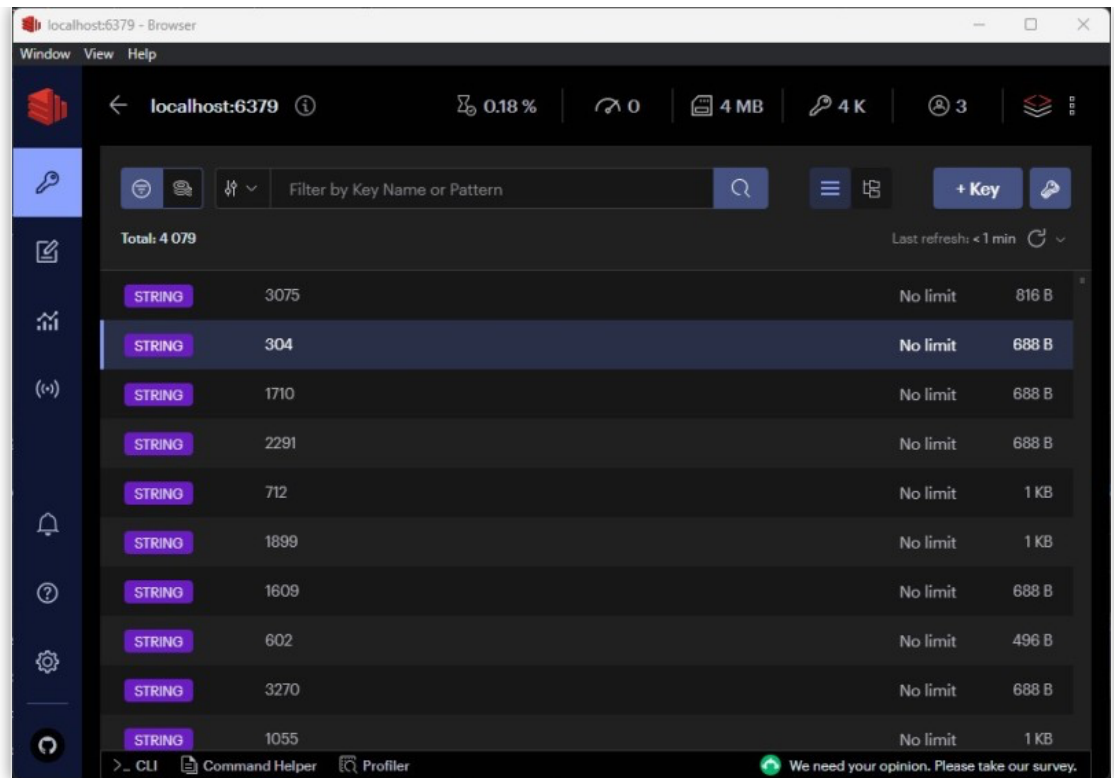
It remains to run and check that there are no errors in the log. Everything worked.

Install redis-insight, look at the data stored in Redis (optional)

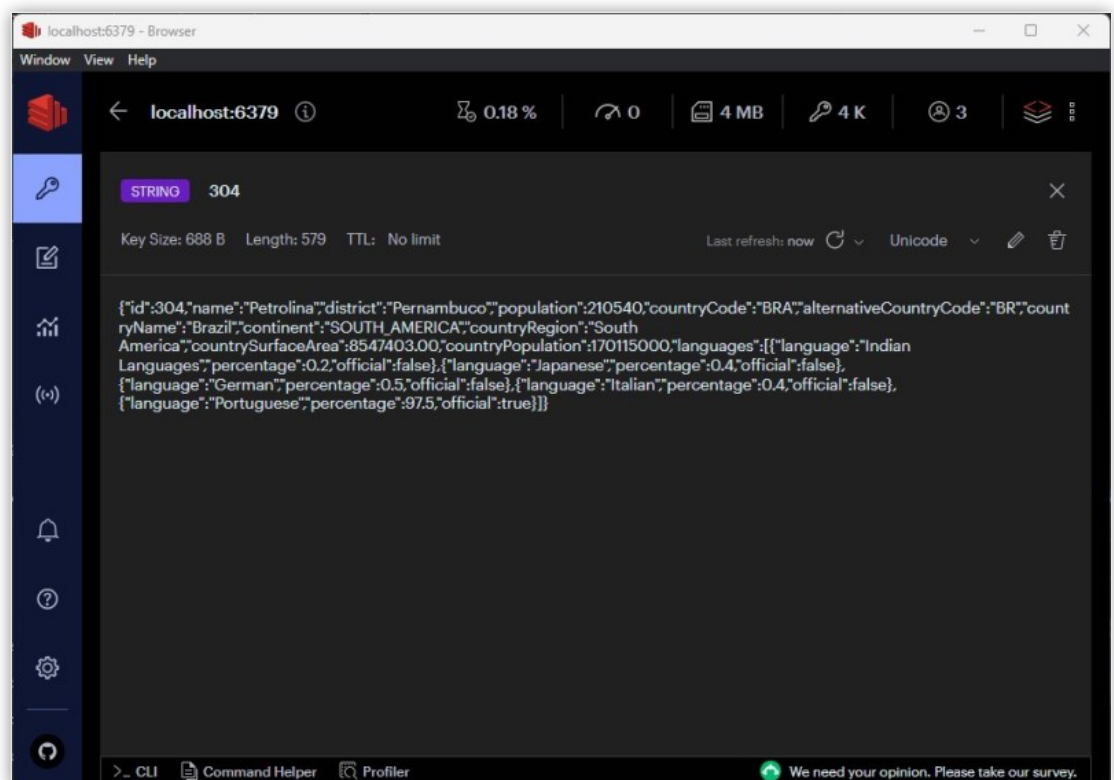
Download redis-insight from [the link](#) and install it. After starting, it immediately shows our radish instance in the docker container:



If you log in, we will see a list of all keys:



And you can go to any key to see what data is stored on it:



Write a method for getting data from Redis

For testing, we use the following test: we get 10 CityCountry records. Each with a separate request, but in one connection.

Data from radish can be obtained through our radish client. To do this, let's write a method that

takes a list of id's to get.

```

1  private void testRedisData(List<Integer> ids) {
2      try (StatefulRedisConnection<String, String> connection = red
3          RedisStringCommands<String, String> sync = connection.syn
4          for (Integer id : ids) {
5              String value = sync.get(String.valueOf(id));
6              try {
7                  mapper.readValue(value, CityCountry.class);
8              } catch (JsonProcessingException e) {
9                  e.printStackTrace();
10             }
11         }
12     }
13 }

```

The implementation, I think, is intuitive: we open a synchronous connection, and for each id we get a JSON String , which we convert into the object of the CityCountry type we need .

Write a method to get data from MySQL

In the CityDAO class, add a method `getId(Integer id)` in which we will get the city along with the country:

```

1  public City getId(Integer id) {
2      Query<City> query = sessionFactory.getCurrentSession().create
3      query.setParameter("ID", id);
4      return query.getSingleResult();
5  }

```

By analogy with the previous paragraph, let's add a similar method for MySQL to the Main class:

```

1  private void testMysqlData(List<Integer> ids) {
2      try (Session session = sessionFactory.getCurrentSession()) {
3          session.beginTransaction();
4          for (Integer id : ids) {
5              City city = cityDAO.getId(id);
6              Set<CountryLanguage> languages = city.getCountry().ge
7          }
8          session.getTransaction().commit();
9      }
10 }

```

Of the features, in order to be sure to get the full object (without proxy stubs), we explicitly request a list of languages from the country.

Compare the speed of getting the same data from MySQL and Redis

Here I will immediately give the code of the main method, and the result that is obtained on my local computer.

```

1  public static void main(String[] args) {
2      Main main = new Main();
3      List<City> allCities = main.fetchData(main);
4      List<CityCountry> preparedData = main.transformData(allCities);
5      main.pushToRedis(preparedData);
6
7      // close the current session in order to make a query to the
8      main.sessionFactory.getCurrentSession().close();
9
10     //choose random 10 id cities
11     //since we did not handle invalid situations, use the existing
12     List<Integer> ids = List.of(3, 2545, 123, 4, 189, 89, 3458, 1
13
14     long startRedis = System.currentTimeMillis();
15     main.testRedisData(ids);
16     long stopRedis = System.currentTimeMillis();
17
18     long startMysql = System.currentTimeMillis();
19     main.testMysqlData(ids);
20     long stopMysql = System.currentTimeMillis();
21
22     System.out.printf("%s:\t%d ms\n", "Redis", (stopRedis - startRedis));
23     System.out.printf("%s:\t%d ms\n", "MySQL", (stopMysql - startMysql));
24
25     main.shutdown();
26 }

```

When testing, there is a feature - data from MySQL is only read, so it can not be restarted between launches of our application. And in Redis they are written.

Although when you try to add a duplicate for the same key, the data will simply be updated, I would recommend that you run the commands to stop the container `docker stop redis-stack` and delete the container between application launches in the terminal `docker rm redis-stack`. After that, raise the container with the radish again `docker run -d --name redis-stack -p 6379:6379 -p 8001:8001 redis/redis-stack:latest` and only after that execute our application.

Here are my test results:

```

;
Redis: 67 ms
MySQL: 95 ms
Dec 14, 2022 2:34:35 PM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl$PoolState stop
INFO: HHH10001008: Cleaning up connection pool [jdbc:psycopg2://localhost:3306/world]

Process finished with exit code 0

```

In total, we have achieved an increase in the performance of the response to the "braking frequent" request by one and a half times. And this is taking into account the fact that in testing we used not the fastest deserialization through **ObjectMapper** . If you change it to GSON, most likely, you can "win" a little more time.

At this moment, I remember [a joke about a programmer and time](#): read and think about how to write and optimize your code.

[< Previous lesson](#)[Next lesson >](#)

Comments

TO VIEW ALL COMMENTS OR TO MAKE A COMMENT,
[GO TO FULL VERSION](#)

LEARN

Registration
Java Course
Help with Tasks
Pricing
Game Projects
Java Syntax

COMMUNITY

Users
Articles
Forum
Chat
Success Stories
Activity
Affiliate Program

**GYM**

CodeGym is an online course for learning Java programming from scratch. This course is a perfect way to master Java for beginners. It contains 1200+ tasks with instant verification and an essential scope of Java fundamentals theory. To help you succeed in education, we've implemented a set of motivational features: quizzes, coding projects, content about efficient learning, and a Java developer's career.

COMPANY

About us
Contacts
Reviews
Press Room
CodeGym for EDU
FAQ

FOLLOW US

...

Support

INTERFACE LANGUAGE

DOWNLOAD APP

English ▼

Google Play

Huawei AppGallery

App Store

"Programmers Are Made, Not Born" © 2025
CodeGym

